

实验四：微信小程序开发

项目	内容
实验名称	微信小程序开发
实验编号	d20301035104、d20301009204
学号	2023210628
姓名	王全洋
班级	软件232
组别	2
技术栈	微信小程序 (JavaScript)

实验目的

- 掌握微信小程序开发流程
- 学会JS开发
- 理解页面路由与数据传递
- 掌握本地存储使用
- 体验AI辅助开发

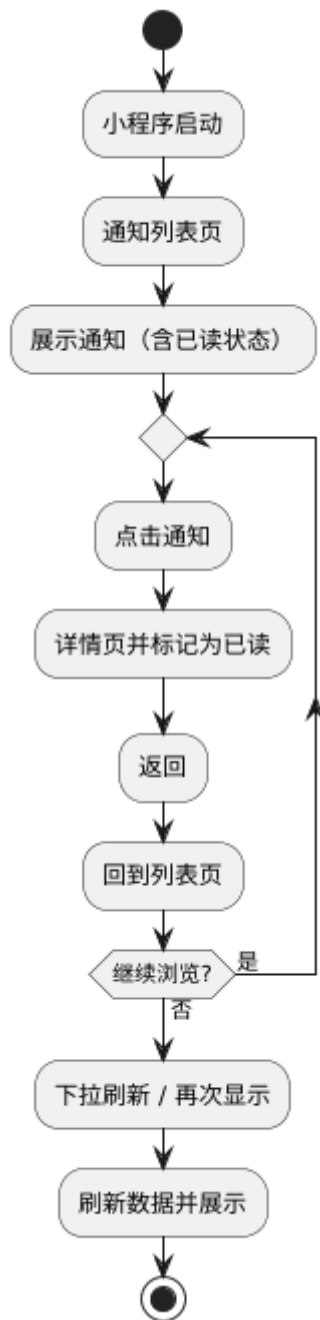
实验环境

工具	版本	用途
微信开发者工具	最新版	小程序开发与调试
Node.js	18	可选，用于npm依赖管理
AI辅助工具	TRAE	代码生成与调试

实验步骤

1. 需求分析

1.1 页面流程图



图片一：小程序页面流程图

说明：页面流程图展示了小程序的整体结构，包括通知列表页（首页）和详情页之间的跳转关系，以及数据传递流程。

详细流程说明：

1. 启动流程

- 用户打开小程序
- 系统加载 app.json 配置文件
- 初始化全局数据（app.js）
- 进入默认首页（通知列表页）

2. 通知列表页（index）

- 页面加载（onLoad）：从静态数据或 API 获取通知列表
- 页面显示（onShow）：重新加载列表，更新已读状态

- 下拉刷新 (onPullDownRefresh) : 重新获取最新数据
- 用户点击某条通知 → 跳转到详情页, 传递通知 ID

3. 通知详情页 (detail)

- 接收参数: 通过 onLoad(options) 获取传递的通知 ID
- 加载详情: 根据 ID 查找对应通知的完整内容
- 标记已读: 将当前通知 ID 存入本地存储 (readIds 数组)
- 页面渲染: 展示标题、发布部门、发布时间、正文内容
- 用户可返回: 点击返回按钮回到列表页

4. 数据流向

- 通知数据: 静态数组存储 (或从 API 获取)
- 已读状态: 使用 uni.setStorageSync 本地存储
- 页面参数: 通过 URL 参数? id=xxx 传递

5. 关键交互

- 列表页 → 详情页: uni.navigateTo 跳转, 传递通知 ID
- 详情页 → 列表页: 系统返回按钮, 自动返回
- 已读状态同步: 列表页 onShow 时重新读取本地存储

1.2 数据模型

```
// 通知数据结构
{
  id: 1,
  title: "关于期末考试安排的通知",
  content: "详细内容...",
  author: "教务处",
  time: "2024-01-01",
  isRead: false
}
```

2. 创建小程序项目

2.1 新建项目

打开微信开发者工具

新建小程序项目

填写AppID (如无则使用测试号)

选择JavaScript模板

3. 通知列表页面开发

3.1 页面模板 (template)

```
<!-- pages/index/index.vue -->
<template>
  <view class="container">
    <view class="news-list">
      <view
        class="news-item"
        :class="{ read: item.isRead }"
        v-for="item in newsList"
        :key="item.id"
        @click="goToDetail(item.id)"
      >
        <view class="title">{{item.title}}</view>
        <view class="info">
          <text>{{item.author}}</text>
          <text>{{item.time}}</text>
        </view>
      </view>
    </view>
  </view>
</template>
```

3.2 样式代码 (style)

```
/* pages/index/index.vue */
.container {
  padding: 0;
  background-color: #f5f5f5;
  min-height: 100vh;
}

.news-list {
  background-color: #fff;
}

.news-item {
  padding: 16px;
  border-bottom: 1px solid #eee;
  background: #fff;
}

.news-item.read {
  opacity: 0.6;
}

.title {
  font-size: 16px;
  font-weight: bold;
  margin-bottom: 8px;
  color: #333;
}
```

```
.info {  
  font-size: 12px;  
  color: #999;  
  display: flex;  
  justify-content: space-between;  
}
```

3.3 JS逻辑 (script)

```
// pages/index/index.vue  
export default {  
  data() {  
    return {  
      newsList: []  
    }  
  },  
  
  onLoad() {  
    this.loadNews();  
  },  
  
  onShow() {  
    this.loadNews();  
  },  
  
  onPullDownRefresh() {  
    this.loadNews();  
    setTimeout(() => {  
      uni.stopPullDownRefresh();  
    }, 500);  
  },  
  
  methods: {  
    loadNews() {  
      const newsList = [  
        { id: 1, title: "关于期末考试安排的通知", content: "本次期末考试安排如下: \n1. 考试时间: 2024年1月15日-1月26日\n2. 考试地点: 各教学楼指定教室\n3. 请同学们提前30分钟到达考场, 携带学生证和考试用品\n4. 考试期间严格遵守考场纪律, 严禁作弊行为\n5. 如有特殊情况无法参加考试, 请提前向教务处申请缓考", author: "教务处", time: "2024-01-01", isRead: false },  
        { id: 2, title: "图书馆开放时间调整", content: "因期末复习需要, 图书馆开放时间调整如下: \n1. 周一至周日: 7:00-23:00\n2. 自习室24小时开放\n3. 电子阅览室开放时间: 8:00-22:00\n4. 请同学们合理安排学习时间, 保持图书馆安静", author: "图书馆", time: "2024-01-02", isRead: false },  
        { id: 3, title: "校园网络维护通知", content: "校园网络将于以下时间进行维护升级: \n1. 维护时间: 2024年1月5日 23:00-次日6:00\n2. 维护范围: 全校区有线网络及WiFi\n3. 维护期间网络将暂时中断, 请提前做好准备\n4. 如有紧急需求, 请联系信息中心: 8888-1234", author: "信息中心", time: "2024-01-03", isRead: false },  
        { id: 4, title: "寒假放假安排通知", content: "根据学校校历安排, 现将寒假放假有关事项通知如下: \n1. 学生放假时间: 2024年1月27日-2月25日\n2. 2月26日正式上课\n3. 留校学生需提前向辅导员申请登记\n4. 宿舍楼将于1月28日进行安全检查", author: "学生处", time: "2024-01-04", isRead: false },  
      ]  
    }  
  }  
}
```

```

        { id: 5, title: "校园招聘会通知", content: "2024届毕业生春季校园招聘会将
于以下时间举行：\n1. 时间：2024年2月28日 9:00-16:00\n2. 地点：学校体育馆\n3. 参会企业：100
余家知名企业\n4. 请毕业生携带个人简历准时参加", author: "就业指导中心", time: "2024-01-
05", isRead: false }
    ];

    const readIds = uni.getStorageSync('readIds') || [];
    newsList.forEach(news => {
        news.isRead = readIds.includes(news.id);
    });

    this.newsList = newsList;
},

goToDetail(id) {
    uni.navigateTo({
        url: `/pages/detail/detail?id=${id}`
    });
}
}
}

```

3.4 配置文件

```

// pages.json
{
  "pages": [
    {
      "path": "pages/index/index",
      "style": {
        "navigationBarTitleText": "智慧校园通知",
        "enablePullDownRefresh": true
      }
    }
  ]
}

```

4. 详情页开发

4.1 页面模板 (template)

```

<!-- pages/detail/detail.vue -->
<template>
  <view class="container">
    <view class="title">{{news.title}}</view>
    <view class="info">
      <text>发布部门：{{news.author}}</text>
      <text>发布时间：{{news.time}}</text>
    </view>
    <view class="content">{{news.content}}</view>
  </view>
</template>

```

4.2 样式代码 (style)

```
/* pages/detail/detail.vue */
.container {
  padding: 20px;
  background-color: #fff;
  min-height: 100vh;
}

.title {
  font-size: 20px;
  font-weight: bold;
  color: #333;
  margin-bottom: 16px;
  line-height: 1.5;
}

.info {
  font-size: 14px;
  color: #999;
  margin-bottom: 20px;
  padding-bottom: 16px;
  border-bottom: 1px solid #eee;
  display: flex;
  flex-direction: column;
  gap: 8px;
}

.content {
  font-size: 16px;
  color: #333;
  line-height: 1.8;
  white-space: pre-line;
}
```

4.3 JS逻辑 (script)

```
// pages/detail/detail.vue
export default {
  data() {
    return {
      news: {}
    }
  },

  onLoad(options) {
    const id = parseInt(options.id);
    this.loadDetail(id);
  },

  methods: {
    loadDetail(id) {
      const allNews = [
```

```

        { id: 1, title: "关于期末考试安排的通知", content: "本次期末考试安排如下：\n1. 考试时间：2024年1月15日-1月26日\n2. 考试地点：各教学楼指定教室\n3. 请同学们提前30分钟到达考场，携带学生证和考试用品\n4. 考试期间严格遵守考场纪律，严禁作弊行为\n5. 如有特殊情况无法参加考试，请提前向教务处申请缓考", author: "教务处", time: "2024-01-01" },
        { id: 2, title: "图书馆开放时间调整", content: "因期末复习需要，图书馆开放时间调整如下：\n1. 周一至周日：7:00-23:00\n2. 自习室24小时开放\n3. 电子阅览室开放时间：8:00-22:00\n4. 请同学们合理安排学习时间，保持图书馆安静", author: "图书馆", time: "2024-01-02" },
        { id: 3, title: "校园网络维护通知", content: "校园网络将于以下时间进行维护升级：\n1. 维护时间：2024年1月5日 23:00-次日6:00\n2. 维护范围：全校区有线网络及WiFi\n3. 维护期间网络将暂时中断，请提前做好准备\n4. 如有紧急需求，请联系信息中心：8888-1234", author: "信息中心", time: "2024-01-03" },
        { id: 4, title: "寒假放假安排通知", content: "根据学校校历安排，现将寒假放假有关事项通知如下：\n1. 学生放假时间：2024年1月27日-2月25日\n2. 2月26日正式上课\n3. 留校学生需提前向辅导员申请登记\n4. 宿舍楼将于1月28日进行安全检查", author: "学生处", time: "2024-01-04" },
        { id: 5, title: "校园招聘会通知", content: "2024届毕业生春季校园招聘会将于以下时间举行：\n1. 时间：2024年2月28日 9:00-16:00\n2. 地点：学校体育馆\n3. 参会企业：100余家知名企业\n4. 请毕业生携带个人简历准时参加", author: "就业指导中心", time: "2024-01-05" }
    ];

    const news = allNews.find(n => n.id === id);
    if (news) {
        this.news = news;
        this.markAsRead(id);
    }
},

markAsRead(id) {
    let readIds = uni.getStorageSync('readIds') || [];
    if (!readIds.includes(id)) {
        readIds.push(id);
        uni.setStorageSync('readIds', readIds);
    }
}
}
}
}

```

4.4 配置文件

```

// pages.json
{
  "pages": [
    {
      "path": "pages/detail/detail",
      "style": {
        "navigationBarTitleText": "通知详情"
      }
    }
  ]
}

```


5. 本地存储功能

5.1 已读状态存储

```
// 保存已读ID
uni.setStorageSync('readIds', [1, 2, 3]);

// 读取已读ID
const readIds = uni.getStorageSync('readIds');

// 检查是否已读
const isRead = readIds.includes(newsId);
```

6. 测试与发布

6.1 真机预览

- 编译项目
- 扫描二维码预览
- 测试各项功能

6.2 体验版发布

- 点击上传
- 填写版本信息
- 获取体验版二维码

实验结果

验证结果

功能	状态	备注
通知列表展示	成功	
详情页跳转	成功	
已读状态存储	成功	
下拉刷新	成功	
真机预览	成功	

截图记录

通知列表页面

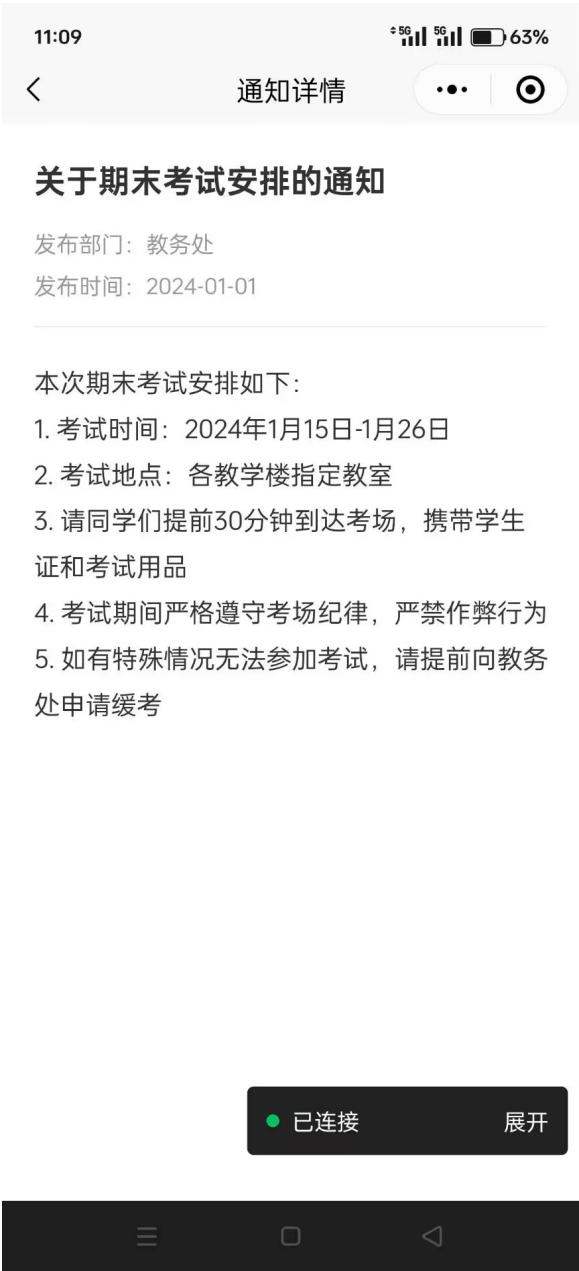


图片二：通知列表页面

说明：通知列表页面展示所有通知的标题、发布部门和时间，已读通知会以半透明效果显示。支持下拉刷新功能。

功能特点：

- 列表按时间倒序排列，最新通知在最上方
- 每条通知显示标题、发布部门、发布时间三个关键信息
- 已读通知透明度降低（opacity: 0.6），便于区分
- 点击任意通知条目跳转到详情页
- 下拉页面可刷新数据



图片三：通知详情页面

说明：通知详情页面展示完整的通知内容，包括标题、发布部门、发布时间和正文内容。正文内容保持原有格式，便于阅读。

功能特点：

- 顶部显示通知标题（大字号加粗）
- 标题下方显示发布部门和发布时间
- 正文内容使用 pre-line 格式，保留换行和缩进
- 页面加载时自动标记为已读
- 支持长按复制文本内容

已读状态效果



图片四：已读状态效果

说明：已读通知在列表中以半透明效果显示，通过本地存储记录已读 ID，下次进入列表页时自动更新显示状态。

技术实现：

- 使用 uni.setStorageSync 存储已读通知 ID 数组
- 列表加载时读取已读 ID，通过 includes 方法判断
- 使用 Vue 的条件类名绑定实现样式切换
- onShow 生命周期确保状态实时更新
- 数据持久化，关闭小程序后仍保留

问题与解决

问题描述	解决方案	解决结果
页面跳转时数据传递问题	使用 onLoad 方法接收参数，通过 options 获取传递的 ID	成功实现页面间数据传递
已读状态刷新问题	在 onShow 生命周期重新加载列表，读取本地存储更新已读状态	列表页能正确显示已读状态
本地存储格式问题	使用数组存储已读 ID，通过 includes 方法判断是否已读	已读状态判断准确
下拉刷新动画不停止	在数据加载完成后调用 uni.stopPullDownRefresh()	下拉刷新功能正常
小程序真机预览失败	检查 AppID 配置，确保在微信公众平台注册	真机预览成功

实验总结

实验收获

- **掌握了微信小程序开发流程：**从项目创建到页面开发，再到测试发布，完整体验了小程序开发的全过程
- **学会了 JS 开发小程序：**使用 JavaScript 编写页面逻辑，理解小程序的数据绑定和事件处理机制
- **理解了页面路由与数据传递：**通过 uni.navigateTo 实现页面跳转，通过参数传递实现页面间数据共享
- **掌握了本地存储使用：**使用 uni.setStorageSync 和 uni.getStorageSync 实现数据的本地持久化存储
- **体验了 AI 辅助开发：**使用 TRAE 等 AI 工具辅助代码生成和调试，提高了开发效率

技术要点

- **小程序页面生命周期：**理解 onLoad、onShow、onPullDownRefresh 等生命周期函数的使用场景
- **本地存储 API：**掌握 uni.setStorageSync 和 uni.getStorageSync 的使用方法
- **页面路由管理：**使用 uni.navigateTo 进行页面跳转，通过 URL 参数传递数据
- **数据绑定与渲染：**使用 Vue 语法进行数据绑定和列表渲染
- **条件样式渲染：**通过: class 动态绑定样式，实现已读/未读状态的视觉区分
- **下拉刷新功能：**配置 enablePullDownRefresh，实现下拉刷新数据加载

改进建议

1. **增加网络请求功能：**将静态数据改为从后端 API 获取，实现真实的数据交互
2. **添加搜索功能：**支持用户搜索通知标题或内容
3. **增加分类筛选：**按通知类型（教务、图书馆、学生处等）进行分类筛选
4. **优化用户体验：**添加加载动画、空状态提示、错误处理等
5. **增加分享功能：**支持将通知分享给好友或朋友圈

6. 添加收藏功能：支持用户收藏重要通知，方便后续查看

附录

项目结构

```
miniprogram/
├─ pages/
│  └─ index/
│     │   └─ index.js           # 列表页逻辑
│     │   └─ index.json        # 列表页配置
│     │   └─ index.wxml        # 列表页模板
│     │   └─ index.wxss        # 列表页样式
│     └─ detail/
│        │   └─ detail.js       # 详情页逻辑
│        │   └─ detail.json     # 详情页配置
│        │   └─ detail.wxml     # 详情页模板
│        │   └─ detail.wxss     # 详情页样式
├─ app.js                       # 小程序入口
├─ app.json                     # 小程序配置
├─ app.wxss                     # 全局样式
└─ project.config.json          # 项目配置
```

核心 API 说明

API	说明	示例
uni.navigateTo	页面跳转	uni.navigateTo({url: '/pages/detail/detail?id=1'})
uni.setStorageSync	同步本地存储	uni.setStorageSync('readIds', [1,2,3])
uni.getStorageSync	同步读取本地存储	const readIds = uni.getStorageSync('readIds')
uni.stopPullDownRefresh	停止下拉刷新	uni.stopPullDownRefresh()
uni.showToast	显示提示框	uni.showToast({title: '加载成功'})

实验感悟：通过本次实验，我深入了解了微信小程序的开发流程和特点。小程序开发与传统 Web 开发有很多相似之处，但也有其独特的地方，如本地存储、页面生命周期等。在实验过程中，我遇到了不少问题，但通过查阅文档和实践探索，最终都得到了解决。这次实验让我对移动应用开发有了更深入的认识，也为今后开发更复杂的小程序应用打下了基础。